arXiv:2402.17152v1 [cs.LG] 27 Feb 2024

Actions Speak Louder than Words: Trillion-Parameter Sequential Transducers for Generative Recommendations

Jiaqi Zhai [1] **Lucy Liao** [1] **Xing Liu** [1] **Yueming Wang** [1] **Rui Li** [1]
Xuan Cao [1] **Leon Gao** [1] **Zhaojie Gong** [1] **Fangda Gu** [1] **Michael He** [1] **Yinghai Lu** [1] **Yu Shi** [1]

Abstract

Large-scale recommendation systems are characterized by their reliance on high cardinality, heterogeneous features and the need to handle tens of billions of user actions on a daily basis. Despite being trained on huge volume of data with thousands of features, most Deep Learning Recommendation Models (DLRMs) in industry fail to scale with compute. Inspired by success achieved by Transformers in language and vision domains, we revisit fundamental design choices in recommendation systems. We reformulate recommendation problems as sequential transduction tasks within a generative modeling framework ("Generative Recommenders"), and propose a new architecture, HSTU, designed for high cardinality, non-stationary streaming recommendation data. HSTU outperforms baselines over synthetic and public datasets by up to 65.8% in NDCG, and is 5.3x to 15.2x faster than FlashAttention2-based Transformers on 8192 length sequences. HSTU-based Generative Recommenders, with 1.5 trillion parameters, improve metrics in online A/B tests by 12.4% and have been deployed on multiple surfaces of a large internet platform with billions of users. More importantly, the model quality of Generative Recommenders empirically scales as a power-law of training compute across three orders of magnitude, up to GPT-3/LLaMa-2 scale, which reduces carbon footprint needed for future model developments, and further paves the way for the first foundational models in recommendations.

1. Introduction

Recommendation systems, quintessential in the realm of online content platforms and e-commerce, play a pivotal role in personalizing billions of user experiences on a daily basis. State-of-the-art approaches in recommendations have been based on Deep Learning Recommendation Models (DLRMs) (Mudigere et al., 2022) for about a decade (Covington et al., 2016; Cheng et al., 2016; Zhou et al., 2018; Tang et al., 2020; Wang et al., 2021; Xia et al., 2023). DLRMs are characterized by their usage of heterogeneous features, such as numerical features – counters and ratios, embeddings, and categorical features such as creator ids, user ids, etc. Due to new content and products being added every minute, the feature space is of extreme high cardinality, often in the range of billions (Eksombatchai et al., 2018). To leverage tens of thousands of such features, DLRMs employ various neural networks to combine features, transform intermediate representations, and finally compose the final outputs.

Despite utilizing extensive human-engineered feature sets and training on vast amounts of data, most DLRMs in industry scale poorly with compute (Zhao et al., 2023). This limitation is noteworthy and remains unanswered.

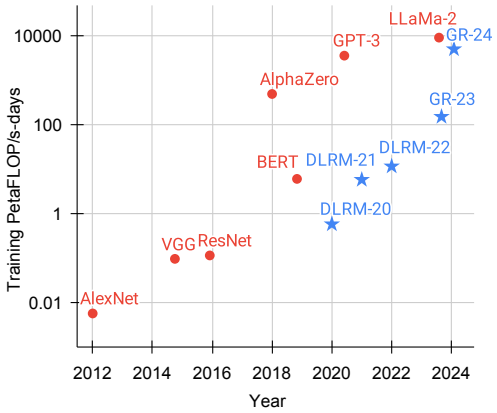


Figure 1. Total compute used to train deep learning models over the years. DLRM results are from (Mudigere et al., 2022); GRs are deployed models from this work. DLRMs/GRs are continuously trained in a streaming setting; we report compute used per year.

Inspired by the success achieved by Transformers in language and vision, we revisit fundamental design choices in modern recommendation systems. We observe that alternative formulations at billion-user scale need to overcome three challenges. First, features in recommendation systems lack explicit structures. While sequential formulations have been explored in small-scale settings (Kang & McAuley,

[1]MRS, Meta AI. Correspondence to: <{jiaqiz, lucyyl, xingl, yuemingw, ruili}@meta.com>.

2018; Sun et al., 2019; Zhou et al., 2020), heterogeneous features, including high cardinality ids, cross features, counters, ratios, etc. play critical roles in industry-scale DLRMs (Mudigere et al., 2022). Second, recommendation systems use billion-scale vocabularies that change continuously. A billion-scale dynamic vocabulary, in contrast to 100K-scale static ones in language (Brown et al., 2020), creates training challenges and necessitates high inference cost given the need to consider tens of thousands of candidates in a target-aware fashion (Zhou et al., 2018; Wang et al., 2020). Finally, computational cost represents the main bottleneck in enabling large-scale sequential models. GPT-3 was trained on a total of 300B tokens over a period of 1-2 months with thousands of GPUs (Brown et al., 2020). This scale appears daunting, until we contrast it with the scale of user actions. The largest internet platforms serve billions of daily active users, who engage with billions of posts, images, and videos per day. User sequences could be of length up to 10^5 in extreme cases (Chang et al., 2023). Consequentially, recommendation systems need to handle a few orders of magnitude more tokens *per day* than what language models process over 1-2 months.

In this work, we treat *user actions* as a new *modality* in generative modeling. Our key insights are, a) core ranking and retrieval tasks in industrial-scale recommenders can be cast as generative modeling problems given an appropriate new feature space; b) this paradigm enables us to systematically leverage redundancies in features, training, and inference to improve efficiency. Due to our new formulation, we deployed models that are *three orders of magnitude* more computationally complex than prior state-of-the-art, while improving topline metrics by 12.4%, as shown in Figure 1.

Our contributions are as follows. We first propose *Generative Recommenders* (GRs) in Section 2, a new paradigm replacing DLRMs. We sequentialize and unify the heterogeneous feature space in DLRMs, with the new approach approximating the full DLRM feature space as sequence length tends to infinity. This enables us to reformulate the main recommendation problems, ranking and retrieval, as pure sequential transduction tasks in GRs. Importantly, this further enables model training to be done in a sequential, generative fashion, which permits us to train on orders of magnitude more data *with the same amount of compute*.

We next address computational cost challenges throughout training and inference. We propose a new sequential transduction architecture, *Hierarchical Sequential Transduction Units* (HSTU). HSTU modifies attention mechanism for large, non-stationary vocabulary, and exploits characteristics of recommendation datasets to achieve 5.3x to 15.2x speedup vs FlashAttention2-based Transformers on 8192 length sequences. Further, through a new algorithm, M-FALCON, that fully amortizes computational costs via micro-batching (Section 3.4), we can serve *285x more complex* GR models while achieving 1.50x-2.48x speedups, all *with the same inference budget* used by traditional DLRMs.

We finally validate the proposed techniques over synthetic datasets, public datasets, and deployments on multiple surfaces of a large internet platform with billions of daily active users in Section 4. To the best of our knowledge, our work represents the first result that shows pure sequential transduction-based architectures, like HSTU, in generative settings (GRs) to significantly outperform DLRMs in large-scale industrial settings. Remarkably, not only did we overcome known scaling bottlenecks in traditional DLRMs, we further succeeded in showing that scaling law (Kaplan et al., 2020) applies to recommendations, representing the potential ChatGPT moment for recommendation systems.

2. Recommendation as Sequential Transduction Tasks: From DLRMs to GRs

2.1. Unifying heterogeneous feature spaces in DLRMs

Modern DLRM models are usually trained with a vast number of categorical ('sparse') and numerical ('dense') features. In GRs, we consolidate and encode these features into a single unified time series, as depicted in Figure 2.

Categorical ('sparse') features. Examples of such features include items that user liked, creators in a category (e.g., Outdoors) that user is following, user languages, communities that user joined, cities from which requests were initiated, etc. We *sequentialize* these features as follows. We first select the longest time series, typically by merging the features that represent items user engaged with, as the main time series. The remaining features are generally time series that slowly change over time, such as demographics or followed creators. We *compress* these time series by keeping the earliest entry per consecutive segment and then merge the results into the main time series. Given these time series change very slowly, this approach does not significantly increase the overall sequence length.

Numerical ('dense') features. Examples of such features include weighted and decayed counters, ratios, etc. For instance, one feature could represent user's past click through rate (CTR) on items matching a given topic. Compared to categorical features, these features change much more frequently, potentially with every single (user, item) interaction. It is therefore infeasible to fully sequentialize such features from computational and storage perspectives. However, an important observation is that the categorical features (e.g., item topics, locations) over which we perform these aggregations are already sequentialized and encoded in GRs. Hence, we can remove numerical features in GRs given a sufficiently expressive sequential transduction architecture *coupled with a target-aware formulation* (Zhou et al., 2018)

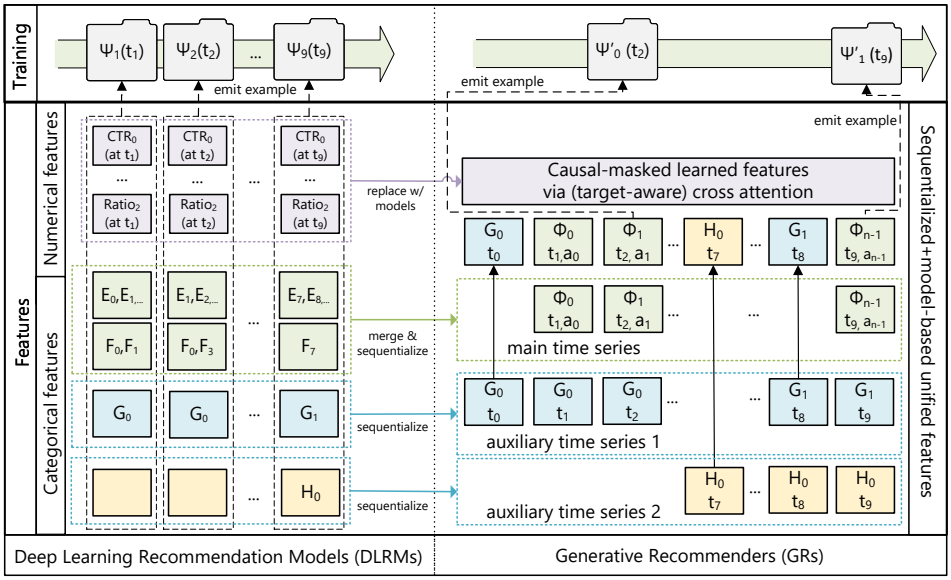


Figure 2. Comparison of features and training procedures: DLRMs vs GRs. E, F, G, H denote categorical features. Φ_i represents the i-th item in the merged main time series. $\Psi_i(t_j)$ denotes training example i emitted at time j.

can meaningfully capture numerical features as we increase the overall sequence length and compute in GRs.

2.2. Reformulating ranking and retrieval as sequential transduction tasks

Given a list of tokens $x_0, x_1, \ldots, x_{n-1}$ ordered chronologically, the time when those tokens are observed $t_0, t_1, \ldots, t_{n-1}$, and other metadata (such as user actions on those tokens a_is, if x_i is a token from the main time series discussed in Section 2.1), a sequential transduction task maps those input sequences to the output tokens $y_0, y_1, \ldots, y_{n-1}$ subject to a mask sequence $m_0, m_1, \ldots, m_{n-1}$ ($m_i \in \{0, 1\}$), where $m_i = 0$ indicates that y_i is undefined. The input tokens come from a dynamic, non-stationary vocabulary $\mathbb{X}$ where the contents (e.g., videos that users engage with) are $\mathbb{X}_c \subseteq \mathbb{X}$. Full notations can be found in Appendix A. We consider causal masked autoregressive settings for these tasks throughout the rest of this section.

Retrieval. In GRs, retrieval tasks learn a distribution $p(x_{i+1}|u_i)$ for each user over $x_{i+1} \in \mathbb{X}_c$, where u_i is the user's representation at step i. A typical objective is to select $\arg\max_{x \in \mathbb{X}_c} p(x|u_i)$ to maximize a specific reward. This differs from a standard autoregressive setup in two ways. First, the supervision for x_i, y_i, is not necessarily x_{i+1}, as users could respond negatively to x_{i+1}. Second, y_i is undefined in situations where the next token represents a non-engagement related categorical feature, such as demographics ($x_{i+1} \notin \mathbb{X}_c$). For these cases, we set $m_i = 0$.

Ranking. Ranking tasks in GRs pose unique challenges as modern recommendation systems often require a "target-aware" formulation. In such a formulation, "interaction" of target, x_{i+1}, and historical features up to i needs to occur as early as possible, which is infeasible with a standard autoregressive setup where "interaction" typically happens late (e.g., via softmax after encoder output). We address this problem by *interleaving* items and actions in the main time series. The resulting new time series (before categorical features $x \notin \mathbb{X}_c$) is then $x_0, a_0, x_1, a_1, \ldots, x_{n-1}, a_{n-1}$, where mask m_is are 0s for the action positions. We apply a small neural network to transform predictions at content positions into multi-task predictions in practice. This approach enables us to apply target-aware cross-attention to all n (user, item) engagements in one pass with causal masking.

2.3. Generative training

Industrial recommenders are commonly trained in a streaming setup, where each example is processed sequentially as they become available. In this setup, the total computational requirement for self-attention based sequential transduction architectures, such as Transformers (Vaswani et al., 2017), scales as $\sum_i n_i(n_i^2 d + n_i d_{ff} d)$, where n_i is the number of tokens of user i, and d is the embedding dimension. The first part in the parentheses comes from self-attention, with assumed $O(n^2)$ scaling factor due to most subquadratic algorithms involving quality tradeoffs and underperforming quadratic algorithms in wall-clock time (Dao et al., 2022). The second part comes from pointwise MLP layers, with hidden layers of size $O(d_{ff}) = O(d)$. Taking $N = \max_i n_i$, the overall time complexity reduces to $O(N^3 d + N^2 d^2)$, which is cost prohibitive for recommendation settings.

To tackle the challenge of training sequential transduction models over long sequences in a scalable manner, we move from traditional impression-level training to *generative training*, reducing the computational complexity by an

$O(N)$ factor, as shown at the top of Figure 2. By doing so, encoder costs are amortized across multiple targets. More specifically, when we sample the i-th user at rate $s_u(n_i)$, the total training cost now scales as $\sum_i s_u(n_i) n_i (n_i^2 d + n_i d^2)$, which is reduced to $O(N^2 d + N d^2)$ by setting $s_u(n_i)$ to $1/n_i$. One way to implement this sampling in industrial-scale systems is to emit training examples at the end of a user's request or session, resulting in $\hat{s_u}(n_i) \propto 1/n_i$.

3. A High Performance Self-Attention Encoder for Generative Recommendations

In order to scale up GRs for industrial-scale recommendation systems with extremely large, non-stationary vocabularies and massive amount of data, we next introduce a new encoder design for GRs called *Hierarchical Sequential Transduction Unit* (HSTU). HSTU is composed of a stack of identical layers connected by residual connections (He et al., 2015). Each layer consists of three main sub-layers: Pointwise Projection (Equation 1), Spatial Aggregation (Equation 2), and Pointwise Transformation (Equation 3):

$$U(X), V(X), Q(X), K(X) = \text{Split}(\phi_1(f_1(X))) \quad (1)$$

$$A(X)V(X) = \phi_2\left(Q(X)K(X)^T + \text{rab}^{p,t}\right)V(X) \quad (2)$$

$$Y(X) = f_2\left(\text{Norm}\left(A(X)V(X)\right) \odot U(X)\right) \quad (3)$$

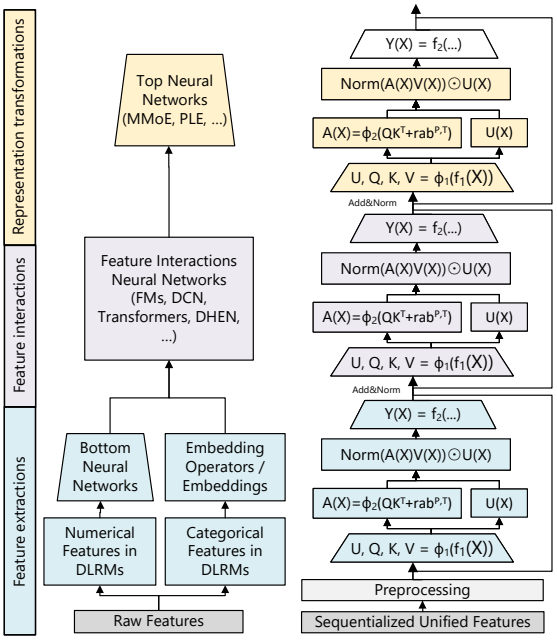


Figure 3. Comparison of key model components: DLRMs vs GRs. The complete DLRM setup (Mudigere et al., 2022) is shown on the left side and a simplified HSTU is shown on the right.

where $f_i(X)$ denotes an MLP; we use one linear layer, $f_i(X) = W_i(X) + b_i$ for f_1 and f_2 to reduce compute complexity; ϕ_1 and ϕ_2 denote nonlinearity, for both of which we use SiLU (Elfwing et al., 2017); and $\text{rab}^{p,t}$ denotes relative attention bias (Raffel et al., 2020) that incorporates positional (p) and temporal (t) information.

HSTU encoder design allows for the replacement of heterogeneous modules in DLRMs with a single modular block. We observe that there are, effectively, three main stages in DLRMs: *Feature Extraction*, *Feature Interactions*, and *Transformations of Representations*. *Feature Extractions* retrieves the pooled embedding representations of categorical features. Their most advanced versions can be generalized as pairwise attention and target-aware pooling (Zhou et al., 2018), which is captured with HSTU layers.

Feature Interaction is the most critical part of DLRMs. Common approaches used include factorization machines and their neural network variants (Rendle, 2010; Guo et al., 2017; Xiao et al., 2017), higher order feature interactions (Wang et al., 2021), and their deep residual variants (Zhang et al., 2022). HSTU replaces feature interactions by enabling attention pooled features to directly "interact" with other features via $\text{Norm}\left(A(X)V(X)\right) \odot U(X)$. This design is motivated by the difficulty of approximating dot products with learned MLPs (Rendle et al., 2020; Zhai et al., 2023a). Given SiLU is applied to $U(X)$, $\text{Norm}\left(A(X)V(X)\right) \odot U(X)$ can also be interpreted as a variant of SwiGLU (Shazeer, 2020).

Transformations of Representations is commonly done with Mixture of Experts (MoEs) and routing to handle diverse, heterogeneous populations. A key idea is to perform conditional computations by specializing sub-networks for different users (Ma et al., 2018; Tang et al., 2020). Element-wise dot products in HSTU can virtually perform gating operations used in MoEs up to a normalization factor.

3.1. Pointwise aggregated attention

HSTU adopts a new pointwise aggregated attention mechanism instead of softmax attention in Transformers. This is motivated by two factors. First, in recommendations, the number of prior data points related to target serves as a strong feature indicating the intensity of user preferences, which is hard to capture after the softmax normalization. This is critical as we need to predict both the intensity of engagements, e.g., time spent on a given item, and the relative ordering of the items, e.g., predicting an ordering of candidates to maximize AUC. Second, while softmax activation is robust to noise by construction, it is less suited for non-stationary vocabularies in streaming settings.

The proposed pointwise aggregated attention mechanism is depicted in Equation (2). Importantly, layer norm is needed after pointwise pooling to stabilize training. One way to understand this design is through synthetic data following a Dirichlet Process that generates streaming data over a

Architecture	HR@10	HR@50
Transformers	.0442	.2025
HSTU (-rab^{p,t}, Softmax)	.0617	.2496
HSTU (-rab^{p,t})	**.0893**	**.3170**

Table 1. Synthetic data in one-pass streaming settings.

nonstationary vocabulary (details in Appendix B). In this setting, we can observe gaps as large as 44.7% between softmax and pointwise attention setups as shown in Table 1.

3.2. Leveraging and algorithmically increasing sparsity

In recommendation systems, the length of user history sequences often follows a skewed distribution, leading to sparse input sequences, particularly in the settings with very long sequences. This sparsity can be leveraged to significantly improve the efficiency of the encoder. To achieve this, we have developed an efficient attention kernel for GPUs that fuses back-to-back GEMMs in a manner similar to FlashAttention (Rabe & Staats, 2021; Dao et al., 2022) but performs *fully raggified* attention computations. This essentially transforms the attention computation into grouped GEMMs of various sizes. As a result, self-attention in HSTU becomes memory-bound and scales as $\Theta(\sum_i n_i^2 d_{qk}^2 R^{-1})$ in terms of memory accesses, where n_i is the sequence length for sample i, d_{qk} is attention dimension, and R is the register size. This approach by itself leads to 2-5x throughput gains as discussed in Section 4.2.

We further *algorithmically* increase the sparsity of user history sequences via *Stochastic Length* (SL). One key characteristic of user history sequences in recommendations is that user behaviors are temporally repetitive, as user behaviors manifest at multiple scales throughout their interaction history. This represents an opportunity to increase sparsity *artificially* without compromising model quality, thereby significantly reducing encoder cost that scales as $\Theta(\sum_i n_i^2)$.

Specifically, let $\Gamma(n, L)$ be a function that selects a subsequence of length L from the original sequence $x_0, \ldots, x_{n-1}$. SL selects input sequences as follows:

$$\begin{aligned} & x_0, \ldots, x_{n_i - 1} \text{ if } n_i \leq N^{\alpha/2} \\ & \Gamma(n_i, N^{\alpha/2}) \text{ if } n_i > N^{\alpha/2}, \text{w/ probability } 1 - N^{\alpha}/n_i^2 \\ & x_0, \ldots, x_{n_i - 1} \text{ if } n_i > N^{\alpha/2}, \text{w/ probability } N^{\alpha}/n_i^2 \end{aligned} \quad (4)$$

which reduces overall attention-related complexity to $O(N^{\alpha} d)$ for $\alpha \in (1, 2]$. We remark that applying SL to training leads to an overall cost-effective model system design, as training stage involves a significantly higher computational cost compared to the inference stage.

Table 2 presents the *sparsity* (see Appendix D) for different sequence lengths and α values, for a representative industry-scale configuration with 30-day user history. The settings that result in negligible regression in model quality are underlined and highlighted in blue. The rows labeled "$\alpha = 2.0$" represents the base sparsity case where SL is not applied. Lower α's are applicable to longer sequences up to the longest sequence length we tested, 8,192.

Alpha (α)	Max Sequence Lengths			
	1,024	2,048	4,096	8,192
1.6	71.5%	76.1%	80.5%	84.4%
1.7	56.1%	63.6%	69.8%	75.6%
1.8	40.2%	45.3%	54.1%	66.4%
1.9	17.2%	21.0%	36.3%	64.1%
2.0	3.1%	6.6%	29.1%	64.1%

Table 2. Impact of *Stochastic Length* (SL) on sequence sparsity.

3.3. Minimizing activation memory usage

In recommendation systems, the use of large batch sizes is crucial for both training throughput (Mudigere et al., 2022) and model quality (Yang et al., 2020; Chen et al., 2020; Zhai et al., 2023a). Consequently, activation memory usage becomes a major scaling bottleneck, in contrast to large language models that are commonly trained with small batch sizes and dominated by parameter memory usage.

Compared to Transformers, HSTU employs a simplified and fully fused design that significantly reduces activation memory usage. Firstly, HSTU reduces the number of linear layers outside of attention from six to two, aligning with recent work that uses elementwise gating to reduce MLP computations (Hua et al., 2022; Gu et al., 2022). Secondly, HSTU aggressively fuses computations into single operators, including $\phi_1(f_1(\cdot))$ in Equation (1), and layer norm, optional dropout, and output MLP in Equation (3). This simplified design reduces the activation memory usage to $2d + 2d + 4hd_{qk} + 4hd_v + 2hd_v = 14d$ per layer in bfloat16.

For comparison, Transformers use a feedforward layer and dropout after attention (intermediate state of $3hd_v$), followed by a pointwise feedforward block consisting of layer norm, linear, activation, linear, and dropout, with intermediate states of $2d + 4d_{ff} + 2d + 1d = 4d + 4d_{ff}$. Here, we make standard assumptions that $hd_v \geq d$ and that $d_{ff} = 4d$ (Vaswani et al., 2017; Brown et al., 2020). Thus, after accounting for input and input layer norm ($4d$) and qkv projections, the total activation states is $33d$. HSTU's design hence enables scaling to > 2x deeper layers.

Additionally, large scale atomic ids used to represent vocabularies also require significant memory usage. With a 10b vocabulary, 512d embeddings, and Adam optimizer, storing embeddings and optimizer states in fp32 already requires 60TB memory. To alleviate memory pressure, we employ rowwise AdamW optimizers (Gupta et al., 2014; Khudia et al., 2021) and place optimizer states on DRAM, which reduces HBM usage per float from 12 bytes to 2 bytes.

3.4. Scaling up inference via cost-amortization

The last challenge we address is the large number of candidates recommendation systems need to process at serving time. We focus on ranking as for retrieval, encoder cost is fully amortizable, and efficient algorithms exist for both MIPS leveraging quantization, hashing, or partitioning (Jegou et al., 2011; Shrivastava & Li, 2014; Li et al., 2002; Zhai et al., 2011) and non-MIPS cases via beam search or hierarchical retrieval (Zhuo et al., 2020; Zhai et al., 2023a).

For ranking, we have up to tens of thousands of candidates (Covington et al., 2016; Wang et al., 2020). We propose an algorithm M-FALCON (Microbatched-Fast Attention Leveraging Cacheable OperatioNs) to perform inference for m candidates with an input sequence size of n.

Within a forward pass, M-FALCON handles b_m candidates in parallel by modifying attention masks and $\text{rab}^{p,t}$ biases such that the attention operations performed for b_m candidates are exactly the same. This reduces the cost of applying cross-attention from $O(b_m n^2 d)$ to $O((n + b_m)^2 d) = O(n^2 d)$ when b_m can be considered a small constant relative to n. We optionally divide the overall m candidates into $\lceil m/b_m \rceil$ microbatches of size b_m to leverage encoder-level KV caching (Pope et al., 2022) either across forward passes to reduce cost, or across *requests* to minimize tail latency.

Overall, M-FALCON enables model complexity to linearly scale up with the number of candidates in traditional DLRMs's ranking stages; we succeeded in applying *a 285x* more complex target-aware cross attention model at 1.5x throughput with a constant inference budget for a typical ranking configuration discussed in Section 4.3.

4. Experiments

4.1. Validating Inductive Hypotheses of HSTU Encoder

4.1.1. Traditional Sequential Settings

We first evaluate the performance of HSTU on two popular recommender datasets, MovieLens and Amazon Reviews. We follow sequential recommendation settings in literature, including *full shuffle* and *multi-epoch* training. For baseline, we use SASRec, a state-of-the-art Transformer implementation (Kang & McAuley, 2018). We report Hit Rate@K and NDCG@K over the entire corpus, consistent with recent work (Dallmann et al., 2021; Zhai et al., 2023a).

Results are presented in Table 3. "SASRec (2023)" denotes the best SASRec recipe reported in (Zhai et al., 2023a). The rows labeled "HSTU" use identical configurations as SASRec (same number of layers, heads, etc.). "HSTU-large" represents larger HSTU encoders (4x number of layers and 2x number of heads). Results show that a) HSTU, with its design optimized for recommendations, significantly outperforms the baseline when using the same configuration, and b) HSTU further improves performance when scaled up.

It is important to note that the evaluation methodology used here differs significantly from industrial-scale settings, as *full-shuffle* and *multi-epoch* training are generally not practical in streaming settings used in industry (Liu et al., 2022).

4.1.2. Industrial-scale Streaming Settings

We next compare the performance of HSTU, ablated HSTUs, and transformers using industrial-scale datasets in a streaming setting. Throughout the rest of this section, we report Normalized Entropy (NE) (He et al., 2014) for ranking. We train the models over 100B examples (DLRM equivalent), with 64-256 H100s used per job. Given ranking is done in a multi-task setting, we report the main engagement event ("E-Task") and the main consumption event ("C-Task"). In our context, we consider a 0.001 reduction in NE significant as it generally leads to .5% topline metric improvements for billions of users. For retrieval, given the setup is similar to language modeling, we report log perplexity. We fix encoder parameters in a smaller-scale setting ($l = 3$, $n = 2048$, $d = 512$ for ranking and $l = 6$, $n = 512$, $d = 256$ for retrieval), and grid-search other hyperparameters due to resource limits.

We show results in Table 4. First, HSTU significantly outperforms Transformers, especially in ranking, likely due to pointwise attention and improved relative attention biases. Second, the gaps between the ablated HSTUs and HSTU confirm the effectiveness of our designs. Optimal learning rates are about 10x lower for Softmax-based HSTU and Transformer vs the rest due to training stability. Even with lower learning rates and pre-norm residual connections (Xiong et al., 2020), we encountered frequent loss explosions with standard Transformers in ranking. Finally, HSTU outperforms a popular Transformer variant used in LLMs, Transformer++ (Touvron et al., 2023a), which uses RoPE (Su et al., 2023), SwiGLU, etc. Overall, in this small-scale setting, HSTU shows better quality at 1.5x-2x faster wall-clock time and uses 50% less HBM usage.

4.2. Encoder Efficiency

Stochastic Length. Figure 4 and Figure 5 (a) show the impact of *stochastic length* (SL) on model metrics. At $\alpha = 1.6$, a sequence of length 4096 is turned into a sequence of length 776 the majority of the time, or removing more than 80% of the tokens. Even after sparsity ratio increases to 64%-84%, the NEs we obtained for main tasks did not degrade by more than 0.002 (0.2%). This evidence supports that SL, for suitable αs, does not negatively impact model quality and allows for high sparsity to reduce training cost. We further verify in Appendix D.3 that SL significantly outperforms existing length extrapolation techniques.

Table 3. Evaluations of methods on public datasets in multi-pass, full-shuffle settings.

	Method	HR@10	HR@50	HR@200	NDCG@10	NDCG@200
ML-1M	SASRec (2023)	.2828	.5508	.7522	.1545	.2441
	HSTU	.3043 (+7.6%)	.5728 (+4.0%)	.7740 (+2.9%)	.1700 (+10.1%)	.2601 (+6.6%)
	HSTU-large	**.3306 (+16.9%)**	**.5897 (+7.1%)**	**.7832 (+4.1%)**	**.1858 (+20.3%)**	**.2730 (+11.9%)**
ML-20M	SASRec (2023)	.2906	.5499	.7655	.1621	.2521
	HSTU	.3252 (+11.9%)	.5885 (+7.0%)	.7943 (+3.8%)	.1878 (+15.9%)	.2774 (+10.0%)
	HSTU-large	**.3567 (+22.8%)**	**.6149 (+11.8%)**	**.8076 (+5.5%)**	**.2106 (+30.0%)**	**.2971 (+17.9%)**
Books	SASRec (2023)	.0292	.0729	.1400	.0156	.0350
	HSTU	.0404 (+38.4%)	.0943 (+29.5%)	.1710 (+22.1%)	.0219 (+40.6%)	.0450 (+28.6%)
	HSTU-large	**.0469 (+60.6%)**	**.1066 (+46.2%)**	**.1876 (+33.9%)**	**.0257 (+65.8%)**	**.0508 (+45.1%)**

Table 4. Evaluation of HSTU, ablated HSTU, and Transformers on industrial-scale datasets in one-pass streaming settings.

Architecture	Retrieval	Ranking (NE)	
	log pplx.	E-Task	C-Task
Transformers	4.069	NaN	NaN
HSTU (-rab^{p,t}, Softmax)	4.024	.5067	.7931
HSTU (-rab^{p,t})	4.021	.4980	.7860
Transformer++	4.015	.4945	.7822
HSTU (original rab)	4.029	.4941	.7817
HSTU	**3.978**	**.4937**	**.7805**

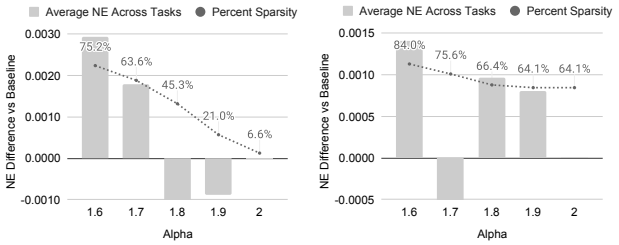


Figure 4. Impact of *Stochastic Length* (SL) on metrics. Left: $n = 4096$. Right: $n = 8192$. Full results can be found in Appendix D.

Encoder Efficiency. Figure 5 compares the efficiency of HSTU and Transformer encoders in training and inference settings. For Transformers, we use the state-of-the-art FlashAttention-2 (Dao, 2023) implementation. We consider sequence lengths ranging from 1,024 to 8,192 and apply *Stochastic Length* (SL) during training. In the evaluation, we use the same configuration for HSTU and Transformer ($d = 512$, $h = 8$, $d_{qk} = 64$) and ablate *relative attention bias* considering HSTU outperforms Transformers without rab^{p,t}, as demonstrated in Section 4.1.2. We compare the encoder-level performance in bfloat16 on NVIDIA H100 GPUs. Overall, HSTU is up to 15.2x and 5.6x more efficient than Transformers in training and inference, respectively.

Additionally, the decrease in activation memory usage as discussed in Section 3.3 enables us to construct over 2x deeper networks with HSTUs compared to Transformers.

4.3. Generative Recommenders vs DLRMs in Industrial-scale Streaming Settings

Lastly, we compare the end-to-end performance of GRs against state-of-the-art DLRM baselines in industrial-scale streaming settings. Our GR implementation reflects a typical configuration used in production, whereas the DLRM configurations reflect continued iterations of hundreds of people over multiple years. Given multiple generators are generally used in the retrieval stage of a recommendation system, we report both the online result for adding GR ("add source") and replacing existing main DLRM source ("replace source"). Table 5 and Table 6 show that GR not only significantly outperforms DLRMs offline, but also brings 12.4% wins in online A/B tests.

Table 5. Offline/Online Comparison of Retrieval Models.

Methods	Offline HR@K		Online metrics	
	K=100	K=500	E-Task	C-Task
DLRM	29.0%	55.5%	+0%	+0%
DLRM (abl. features)	28.3%	54.3%	–	
GR (content-based)	11.6%	18.8%	–	
GR (interactions only)	35.6%	61.7%	–	
GR (new source)	**36.9%**	**62.4%**	**+6.2%**	**+5.0%**
GR (replace source)			**+5.1%**	**+1.9%**

Table 6. Offline/Online Comparison of Ranking Models.

Methods	Offline NEs		Online metrics	
	E-Task	C-Task	E-Task	C-Task
DLRM	.4982	.7842	+0%	+0%
DLRM (abl. features)	.5053	.7925	–	–
GR (interactions only)	.4851	.7903	–	–
GR	**.4845**	**.7645**	**+12.4%**	**+4.4%**

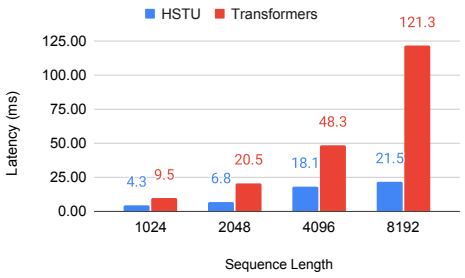
As discussed in Section 2, GRs build upon raw categorical engagement features, while DLRMs are typically trained with a significantly larger number of features, the majority of which are handcrafted from raw signals. If we give the same set of features used in GRs to DLRMs ("DLRM (abl. features)"), the performance of DLRMs is significantly degraded, which suggests GRs can meaningfully capture those features via their architecture and unified feature space.

We further validate the GR formulation in Section 2.2 by comparing it with a traditional sequential recommender setup that only considers items user interacted with (Kang & McAuley, 2018) ("GR (interactions only)"). The results are significantly worse, with its ranking variant underperforming GRs by 2.6% in NE in the main consumption task.

Considering the popularity of content-based methods (in-

(a) Training NE.

(b) Training Speedup.



(c) Inference Speedup.

Figure 5. Encoder-level efficiency: HSTU vs FlashAttention2-based Transformers for Training (a, b) and Inference (c).

cluding LMs), we also include a GR baseline with only content features ("GR (content-based)"). The substantial gap in performance of content-based baselines and DLRMs/GRs underscores the significance of high cardinality user actions.

We finally compare the efficiency of GRs with our production DLRMs in Figure 6. Despite the GR model being 285x more complex in terms of FLOPs, we achieved 1.50x/2.48x higher throughput when scoring 1024/16384 candidates, respectively, due to the efficient HSTU encoder design and the M-FALCON serving algorithm presented in Section 3.4.

4.3.1. Scaling Law for Recommendation Systems

It is commonly known that in large-scale industrial settings, DLRMs saturate in quality at certain compute and params regimes (Zhao et al., 2023). We compare the scalability of GRs and DLRMs to better understand this phenomenon.

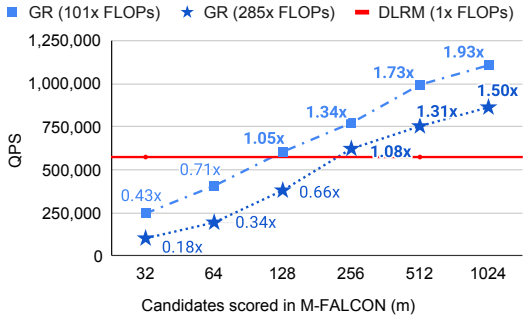


Figure 6. Comparison of inference throughput, in the most challenging ranking setup. Full results can be found in Appendix E.

Since feature interaction layers are crucial for DLRM's performance (Mudigere et al., 2022), we experimented with Transformers (Vaswani et al., 2017), DHEN (Zhang et al., 2022), and a variant of DCN (Wang et al., 2021) augmented with residual connections (He et al., 2015) used in our production settings to scale up the DLRM baseline in the ranking setting. For the retrieval baseline, given our baseline used a residual setup, we scaled up hidden layer sizes, embedding dimensions, and number of layers. For HSTU-based Generative Recommenders (GRs), we scaled up the model by adjusting the hyperparameters for HSTU, including the number of residual layers, sequence length, embedding dimensions, number of attention heads, etc. We additionally adjust the number of negatives for retrieval.

Results are shown in Figure 7. In the low compute regime, DLRMs might outperform GRs due to handcrafted features, corroborating the importance of feature engineering in traditional DLRMs. However, GRs demonstrate substantially better scalability with respect to FLOPs when continuing to increase the model complexity, whereas DLRM performance plateaus, consistent with findings in prior work. We also observe better scalability w.r.t. both embedding parameters and non-embedding parameters, with GRs leading to 1.5 trillion parameter models, whereas DLRMs performance saturate at about 200 billion parameters.

Finally, all of our main metrics, including Hit Rate@100 and Hit Rate@500 for retrieval, and NE for ranking, empirically scale as a power law of compute used given appropriate hyperparameters. We observe this phenomenon across three orders of magnitude, up till the largest models we were able to test (8,192 sequence length, 1,024 embedding dimension, 24 layers of HSTU), at which point the total amount of compute we used (normalized over 365 days as we use a standard streaming training setting) is close to the total training compute used by GPT-3 (Brown et al., 2020) and LLaMa-2 (Touvron et al., 2023b), as shown in Figure 1. Within a reasonable range, the exact model hyperparameters play less important roles compared to the total amount of training compute applied. In contrast to language modeling (Kaplan et al., 2020), sequence length play a significantly more important role in GRs, and it's important to scale up sequence

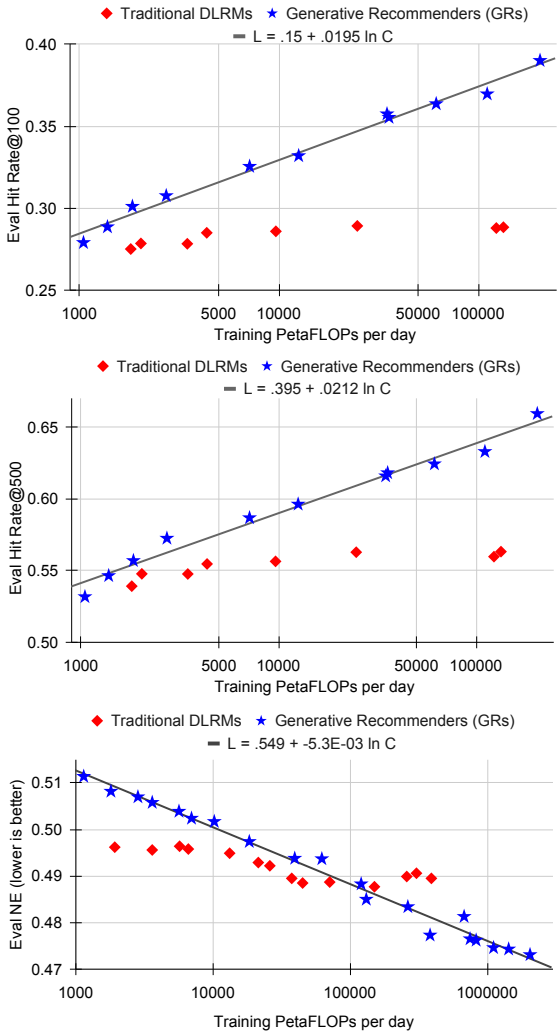


Figure 7. Scalability: DLRMs vs GRs in large-scale industrial settings across retrieval (top, middle) and ranking (bottom). +0.005 in HR and -0.001 in NE represent significant improvements.

length and other parameters (e.g., embedding dimension) in tandem. This is perhaps the most important advantage of our proposed method, as we've shown for the first time that scaling law from LLMs (Kaplan et al., 2020) may also apply to large-scale recommendation systems.

5. Related Work

Prior work on sequential recommenders assumes that user interactions can be reduced to a single homogeneous sequence over items (Kang & McAuley, 2018; Sun et al., 2019). Industrial-scale applications of sequential approaches are primarily pairwise attention (Zhou et al., 2018) or sequential encoders as part of DLRMs (Chen et al., 2019; Xia et al., 2023). Multi-stage attention has been explored in lieu of self-attention to improve efficiency (Chang et al., 2023). Generative approaches that represent ids as a token series have been explored in retrieval (Zhuo et al., 2020).

Efficient attention has been a major research focus area due to self-attention's $O(n^2)$ scaling factor, with major work like factorized attentions (Child et al., 2019), low-rank approximations (Katharopoulos et al., 2020), etc. Recently, alternative formulations for sequential transduction settings have been explored (Gu et al., 2022; Hua et al., 2022). HSTU's elementwise gating design, in particular, is inspired by FLASH (Hua et al., 2022). Recent hardware-aware formulations have been shown to significantly reduce memory usage (Rabe & Staats, 2021; Korthikanti et al., 2022; Zhai et al., 2023b) and give significantly better wallclock time results (Dao et al., 2022). Length extrapolation enables models trained on shorter sequences to generalize, though most work focuses on finetuning or improving bias mechanisms (Press et al., 2022). Our work instead introduces stochasticity in the length dimension, inspired by work on stochasticity in the depth dimension (Huang et al., 2016).

Interests in large language models (LLMs) have motivated work to treat various recommendation tasks as in-context learning (Sileo et al., 2022), instruction tuning (Bao et al., 2023), or transfer learning (Li et al., 2023) on top of pre-trained LLMs. World knowledge embedded in LLMs can be transferred to downstream tasks (Cui et al., 2022) and improve recommendations in zero-shot or few-shot cases. Textual representations of user behavior sequences have also demonstrated good scaling behaviors on medium-scale datasets (Shin et al., 2023). Most studies of LLMs for recommendation have been centered around low-data regimes; in large-scale settings, they have yet to outperform collaborative filtering on MovieLens (Hou et al., 2024).

6. Conclusions

We have proposed Generative Recommenders (GRs), a new paradigm that formulates ranking and retrieval as sequential transduction tasks, allowing them to be trained in a generative manner. This is made possible by the novel HSTU encoder design, which is 5.3x-15.2x faster than state-of-the-art Transformers on 8192 length sequences, and through the use of new training and inference algorithms such as M-FALCON. With GRs, we deployed models that are 285x more complex using similar inference resources. GRs and HSTU have led to 12.4% metric improvements in production and have shown superior scaling performance compared to traditional DLRMs. Our results corroborate that user actions represent an underexplored modality in generative modeling – to echo our title, *"Actions speak louder than words"*.

The dramatic simplification of features in our work paves the way for the first foundational models for recommendations, search, and ads by enabling a unified feature space to be used across domains. The fully sequential setup of GRs also enables recommendation to be formulated in an end-to-end, generative setting. Both of these enable recommendation systems to better assist users holistically.

BROADER IMPACT

We believe that our work has broad positive implications. Reducing reliance of recommendation, search, and ads systems on the large number of heterogeneous features can make these systems much more privacy friendly while improving user experiences. Enabling recommendation systems to attribute users' long term outcomes to short-term decisions via fully sequential formulations could reduce the prevalence of content that do not serve users' long term goals (including clickbaits and fake news) across the web, and better align incentives of platforms with user values. Finally, applications of foundational models and scaling law can help reduce carbon footprints incurred with model research and developments needed for recommendations, search, and related use cases.

Acknowledgements

This work represents the joint efforts of hundreds of people, and would not be possible without work from the following contributors (alphabetical order): Adnan Akhundov, Bugra Akyildiz, Shabab Ayub, Alex Bao, Renqin Cai, Jennifer Cao, Guoqiang Jerry Chen, Lei Chen, Sean Chen, Huihui Cheng, Weiwei Chu, Ted Cui, Shiyan Deng, Nimit Desai, Fei Ding, Francois Fagan, Lu Fang, Liang Guo, Liz Guo, Jeevan Gyawali, Yuchen Hao, Daisy Shi He, Jie Hua, Yanzun Huang, Hongyi Jia, Rui Jian, Jian Jin, Rahul Kindi, Changkyu Kim, Fu Li, Hong Li, Shen Li, Wei Li, Zhijing Li, Xueting Liao, Emma Lin, Hao Lin, Jingzhou Liu, Xingyu Liu, Kai Londenberg, Liang Luo, Linjian Ma, Matt Ma, Yun Mao, Bert Maher, Matthew Murphy, Satish Nadathur, Min Ni, Jongsoo Park, Jing Qian, Lijing Qin, Alex Singh, Timothy Shi, Dennis van der Staay, Xiao Sun, Colin Taylor, Shin-Yeh Tsai, Rohan Varma, Omkar Vichare, Alyssa Wang, Pengchao Wang, Shengzhi Wang, Wenting Wang, Xiaolong Wang, Zhiyong Wang, Wei Wei, Bin Wen, Eric Xu, Bi Xue, Zheng Yan, Chao Yang, Junjie Yang, Zimeng Yang, Chunxing Yin, Daniel Yin, Yiling You, Keke Zhai, Yanli Zhao, Zhuoran Zhao, Hui Zhang, Jingjing Zhang, Lu Zhang, Lujia Zhang, Na Zhang, Rui Zhang, Xiong Zhang, Ying Zhang, Zhiyun Zhang, Charles Zheng, Erheng Zhong, Xin Zhuang. We would like to thank Shikha Kapoor, Rex Cheung, Lana Dam, Ram Ramanathan, Nipun Mathur, Bo Feng, Yanhong Wu, Zhaohui Guo, Hongjie Bai, Wen-Yun Yang, Zellux Wang, Arun Singh, Bruce Deng, Yisong Song, Haotian Wu, Meihong Wang for product support, and Joseph Laria, Akshay Hegde, Abha Jain, Raj Ganapathy for assistance with program management. Finally, we would like to thank Ajit Mathews, Shilin Ding, Hong Yan, Lars Backstrom for their leadership support, and insightful discussions with Andrew Tulloch, Liang Xiong, Kaushik Veeraraghavan, and Gaofeng Zhao.

References

Bao, K., Zhang, J., Zhang, Y., Wang, W., Feng, F., and He, X. Tallrec: An effective and efficient tuning framework to align large language model with recommendation. In *Proceedings of the 17th ACM Conference on Recommender Systems*, RecSys '23. ACM, September 2023. doi: 10.1145/3604915.3608857. URL http://dx.doi.org/10.1145/3604915.3608857.

Brown, T. B., Mann, B., Ryder, N., Subbiah, M., Kaplan, J., Dhariwal, P., Neelakantan, A., Shyam, P., Sastry, G., Askell, A., Agarwal, S., Herbert-Voss, A., Krueger, G., Henighan, T., Child, R., Ramesh, A., Ziegler, D. M., Wu, J., Winter, C., Hesse, C., Chen, M., Sigler, E., Litwin, M., Gray, S., Chess, B., Clark, J., Berner, C., McCandlish, S., Radford, A., Sutskever, I., and Amodei, D. Language models are few-shot learners. 2020.

Chang, J., Zhang, C., Fu, Z., Zang, X., Guan, L., Lu, J., Hui, Y., Leng, D., Niu, Y., Song, Y., and Gai, K. Twin: Two-stage interest network for lifelong user behavior modeling in ctr prediction at kuaishou, 2023.

Chen, Q., Zhao, H., Li, W., Huang, P., and Ou, W. Behavior sequence transformer for e-commerce recommendation in alibaba. In *Proceedings of the 1st International Workshop on Deep Learning Practice for High-Dimensional Sparse Data*, DLP-KDD '19, New York, NY, USA, 2019. Association for Computing Machinery. ISBN 9781450367837. doi: 10.1145/3326937.3341261. URL https://doi.org/10.1145/3326937.3341261.

Chen, T., Kornblith, S., Norouzi, M., and Hinton, G. A simple framework for contrastive learning of visual representations. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20, 2020.

Cheng, H.-T., Koc, L., Harmsen, J., Shaked, T., Chandra, T., Aradhye, H., Anderson, G., Corrado, G., Chai, W., Ispir, M., Anil, R., Haque, Z., Hong, L., Jain, V., Liu, X., and Shah, H. Wide & deep learning for recommender systems. In *Proceedings of the 1st Workshop on Deep Learning for Recommender Systems*, DLRS 2016, pp. 7–10, 2016. ISBN 9781450347952.

Child, R., Gray, S., Radford, A., and Sutskever, I. Generating long sequences with sparse transformers. *CoRR*, abs/1904.10509, 2019. URL http://arxiv.org/abs/1904.10509.

Covington, P., Adams, J., and Sargin, E. Deep neural networks for youtube recommendations. In *Proceedings of the 10th ACM Conference on Recommender Systems*, RecSys '16, pp. 191–198, 2016. ISBN 9781450340359.

Cui, Z., Ma, J., Zhou, C., Zhou, J., and Yang, H. M6-rec: Generative pretrained language models are open-ended recommender systems, 2022.

Dallmann, A., Zoller, D., and Hotho, A. A case study on sampling strategies for evaluating neural sequential item recommendation models. In *Proceedings of the 15th ACM Conference on Recommender Systems*, RecSys '21, pp. 505–514, 2021. ISBN 9781450384582.

Dao, T. Flashattention-2: Faster attention with better parallelism and work partitioning, 2023.

Dao, T., Fu, D. Y., Ermon, S., Rudra, A., and Ré, C. FlashAttention: Fast and memory-efficient exact attention with IO-awareness. In *Advances in Neural Information Processing Systems*, 2022.

Eksombatchai, C., Jindal, P., Liu, J. Z., Liu, Y., Sharma, R., Sugnet, C., Ulrich, M., and Leskovec, J. Pixie: A system for recommending 3+ billion items to 200+ million users in real-time. In *Proceedings of the 2018 World Wide Web Conference*, WWW '18, pp. 1775–1784, 2018. ISBN 9781450356398.

Elfwing, S., Uchibe, E., and Doya, K. Sigmoid-weighted linear units for neural network function approximation in reinforcement learning. *CoRR*, abs/1702.03118, 2017. URL http://arxiv.org/abs/1702.03118.

Gu, A., Goel, K., and Ré, C. Efficiently modeling long sequences with structured state spaces. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL https://openreview.net/forum?id=uYLFoz1vlAC.

Guo, H., Tang, R., Ye, Y., Li, Z., and He, X. Deepfm: A factorization-machine based neural network for ctr prediction. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, IJCAI'17, pp. 1725–1731, 2017. ISBN 9780999241103.

Gupta, M. R., Bengio, S., and Weston, J. Training highly multiclass classifiers. *J. Mach. Learn. Res.*, 15(1): 1461–1492, jan 2014. ISSN 1532-4435.

He, K., Zhang, X., Ren, S., and Sun, J. Deep residual learning for image recognition. *arXiv preprint arXiv:1512.03385*, 2015.

He, X., Pan, J., Jin, O., Xu, T., Liu, B., Xu, T., Shi, Y., Atallah, A., Herbrich, R., Bowers, S., and Candela, J. Q. Practical lessons from predicting clicks on ads at facebook. In *ADKDD'14: Proceedings of the Eighth International Workshop on Data Mining for Online Advertising*, New York, NY, USA, 2014. Association for Computing Machinery. ISBN 9781450329996.

Hou, Y., Zhang, J., Lin, Z., Lu, H., Xie, R., McAuley, J., and Zhao, W. X. Large language models are zero-shot rankers for recommender systems. In *Advances in Information Retrieval - 46th European Conference on IR Research, ECIR 2024*, 2024.

Hua, W., Dai, Z., Liu, H., and Le, Q. V. Transformer quality in linear time. In Chaudhuri, K., Jegelka, S., Song, L., Szepesvári, C., Niu, G., and Sabato, S. (eds.), *International Conference on Machine Learning, ICML 2022, 17-23 July 2022, Baltimore, Maryland, USA*, volume 162 of *Proceedings of Machine Learning Research*, pp. 9099–9117. PMLR, 2022. URL https://proceedings.mlr.press/v162/hua22a.html.

Huang, G., Sun, Y., Liu, Z., Sedra, D., and Weinberger, K. Deep networks with stochastic depth, 2016.

Jegou, H., Douze, M., and Schmid, C. Product quantization for nearest neighbor search. *IEEE Trans. Pattern Anal. Mach. Intell.*, 33(1):117–128, jan 2011. ISSN 0162-8828. doi: 10.1109/TPAMI.2010.57. URL https://doi.org/10.1109/TPAMI.2010.57.

Kang, W.-C. and McAuley, J. Self-attentive sequential recommendation. In *2018 International Conference on Data Mining (ICDM)*, pp. 197–206, 2018.

Kaplan, J., McCandlish, S., Henighan, T., Brown, T. B., Chess, B., Child, R., Gray, S., Radford, A., Wu, J., and Amodei, D. Scaling laws for neural language models. *CoRR*, abs/2001.08361, 2020. URL https://arxiv.org/abs/2001.08361.

Katharopoulos, A., Vyas, A., Pappas, N., and Fleuret, F. Transformers are rnns: Fast autoregressive transformers with linear attention. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

Khudia, D., Huang, J., Basu, P., Deng, S., Liu, H., Park, J., and Smelyanskiy, M. Fbgemm: Enabling high-performance low-precision deep learning inference. *arXiv preprint arXiv:2101.05615*, 2021.

Korthikanti, V., Casper, J., Lym, S., McAfee, L., Andersch, M., Shoeybi, M., and Catanzaro, B. Reducing activation recomputation in large transformer models, 2022.

Li, C., Chang, E., Garcia-Molina, H., and Wiederhold, G. Clustering for approximate similarity search in high-dimensional spaces. *IEEE Transactions on Knowledge and Data Engineering*, 14(4):792–808, 2002.

Li, J., Wang, M., Li, J., Fu, J., Shen, X., Shang, J., and McAuley, J. Text is all you need: Learning language representations for sequential recommendation. In *KDD*, 2023.

Liu, Z., Zou, L., Zou, X., Wang, C., Zhang, B., Tang, D., Zhu, B., Zhu, Y., Wu, P., Wang, K., and Cheng, Y. Monolith: Real time recommendation system with collisionless embedding table, 2022.

Ma, J., Zhao, Z., Yi, X., Chen, J., Hong, L., and Chi, E. H. Modeling task relationships in multi-task learning with multi-gate mixture-of-experts. KDD '18, 2018.

Mudigere, D., Hao, Y., Huang, J., Jia, Z., Tulloch, A., Sridharan, S., Liu, X., Ozdal, M., Nie, J., Park, J., Luo, L., Yang, J. A., Gao, L., Ivchenko, D., Basant, A., Hu, Y., Yang, J., Ardestani, E. K., Wang, X., Komuravelli, R., Chu, C.-H., Yilmaz, S., Li, H., Qian, J., Feng, Z., Ma, Y., Yang, J., Wen, E., Li, H., Yang, L., Sun, C., Zhao, W., Melts, D., Dhulipala, K., Kishore, K., Graf, T., Eisenman, A., Matam, K. K., Gangidi, A., Chen, G. J., Krishnan, M., Nayak, A., Nair, K., Muthiah, B., khorashadi, M., Bhattacharya, P., Lapukhov, P., Naumov, M., Mathews, A., Qiao, L., Smelyanskiy, M., Jia, B., and Rao, V. Software-hardware co-design for fast and scalable training of deep learning recommendation models. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*, ISCA '22, pp. 993–1011, New York, NY, USA, 2022. Association for Computing Machinery. ISBN 9781450386104. doi: 10.1145/3470496.3533727. URL https://doi.org/10.1145/3470496.3533727.

Peng, B., Quesnelle, J., Fan, H., and Shippole, E. YaRN: Efficient context window extension of large language models. In *The Twelfth International Conference on Learning Representations*, 2024. URL https://openreview.net/forum?id=wHBfxhZu1u.

Pope, R., Douglas, S., Chowdhery, A., Devlin, J., Bradbury, J., Levskaya, A., Heek, J., Xiao, K., Agrawal, S., and Dean, J. Efficiently scaling transformer inference, 2022.

Press, O., Smith, N. A., and Lewis, M. Train short, test long: Attention with linear biases enables input length extrapolation. In *The Tenth International Conference on Learning Representations, ICLR 2022, Virtual Event, April 25-29, 2022*. OpenReview.net, 2022. URL https://openreview.net/forum?id=R8sQPpGCv0.

Rabe, M. N. and Staats, C. Self-attention does not need $o(n^2)$ memory, 2021.

Raffel, C., Shazeer, N., Roberts, A., Lee, K., Narang, S., Matena, M., Zhou, Y., Li, W., and Liu, P. J. Exploring the limits of transfer learning with a unified text-to-text transformer. *J. Mach. Learn. Res.*, 21(1), jan 2020. ISSN 1532-4435.

Rendle, S. Factorization machines. In *2010 IEEE International Conference on Data Mining (ICDM)*, pp. 995–1000, 2010. doi: 10.1109/ICDM.2010.127.

Rendle, S., Krichene, W., Zhang, L., and Anderson, J. Neural collaborative filtering vs. matrix factorization revisited. In *Fourteenth ACM Conference on Recommender Systems (RecSys'20)*, pp. 240–248, 2020. ISBN 9781450375832.

Shazeer, N. Glu variants improve transformer, 2020.

Shin, K., Kwak, H., Kim, S. Y., Ramström, M. N., Jeong, J., Ha, J.-W., and Kim, K.-M. Scaling law for recommendation models: towards general-purpose user representations. In *Proceedings of the Thirty-Seventh AAAI Conference on Artificial Intelligence and Thirty-Fifth Conference on Innovative Applications of Artificial Intelligence and Thirteenth Symposium on Educational Advances in Artificial Intelligence*, AAAI'23/IAAI'23/EAAI'23. AAAI Press, 2023. ISBN 978-1-57735-880-0. doi: 10.1609/aaai.v37i4.25582. URL https://doi.org/10.1609/aaai.v37i4.25582.

Shrivastava, A. and Li, P. Asymmetric lsh (alsh) for sublinear time maximum inner product search (mips). In *Advances in Neural Information Processing Systems*, volume 27, 2014.

Sileo, D., Vossen, W., and Raymaekers, R. Zero-shot recommendation as language modeling. In Hagen, M., Verberne, S., Macdonald, C., Seifert, C., Balog, K., Nørvåg, K., and Setty, V. (eds.), *Advances in Information Retrieval - 44th European Conference on IR Research, ECIR 2022, Stavanger, Norway, April 10-14, 2022, Proceedings, Part II*, volume 13186 of *Lecture Notes in Computer Science*, pp. 223–230. Springer, 2022. doi: 10.1007/978-3-030-99739-7_26. URL https://doi.org/10.1007/978-3-030-99739-7_26.

Su, J., Lu, Y., Pan, S., Murtadha, A., Wen, B., and Liu, Y. Roformer: Enhanced transformer with rotary position embedding, 2023.

Sun, F., Liu, J., Wu, J., Pei, C., Lin, X., Ou, W., and Jiang, P. Bert4rec: Sequential recommendation with bidirectional encoder representations from transformer. In *Proceedings of the 28th ACM International Conference on Information and Knowledge Management*, CIKM '19, pp. 1441–1450, 2019. ISBN 9781450369763.

Tang, H., Liu, J., Zhao, M., and Gong, X. Progressive layered extraction (ple): A novel multi-task learning (mtl) model for personalized recommendations. In *Proceedings of the 14th ACM Conference on Recommender Systems*, RecSys '20, pp. 269–278, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450375832. doi: 10.1145/3383313.3412236. URL https://doi.org/10.1145/3383313.3412236.

Touvron, H., Lavril, T., Izacard, G., Martinet, X., Lachaux, M.-A., Lacroix, T., Rozière, B., Goyal, N., Hambro, E., Azhar, F., Rodriguez, A., Joulin, A., Grave, E., and Lample, G. Llama: Open and efficient foundation language models, 2023a.

Touvron, H., Martin, L., Stone, K., Albert, P., Almahairi, A., Babaei, Y., Bashlykov, N., Batra, S., Bhargava, P., Bhosale, S., Bikel, D., Blecher, L., Ferrer, C. C., Chen, M., Cucurull, G., Esiobu, D., Fernandes, J., Fu, J., Fu, W., Fuller, B., Gao, C., Goswami, V., Goyal, N., Hartshorn, A., Hosseini, S., Hou, R., Inan, H., Kardas, M., Kerkez, V., Khabsa, M., Kloumann, I., Korenev, A., Koura, P. S., Lachaux, M.-A., Lavril, T., Lee, J., Liskovich, D., Lu, Y., Mao, Y., Martinet, X., Mihaylov, T., Mishra, P., Molybog, I., Nie, Y., Poulton, A., Reizenstein, J., Rungta, R., Saladi, K., Schelten, A., Silva, R., Smith, E. M., Subramanian, R., Tan, X. E., Tang, B., Taylor, R., Williams, A., Kuan, J. X., Xu, P., Yan, Z., Zarov, I., Zhang, Y., Fan, A., Kambadur, M., Narang, S., Rodriguez, A., Stojnic, R., Edunov, S., and Scialom, T. Llama 2: Open foundation and fine-tuned chat models, 2023b.

Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. Attention is all you need. In *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, pp. 6000–6010, 2017. ISBN 9781510860964.

Wang, R., Shivanna, R., Cheng, D., Jain, S., Lin, D., Hong, L., and Chi, E. Dcn v2: Improved deep & cross network and practical lessons for web-scale learning to rank systems. In *Proceedings of the Web Conference 2021*, WWW '21, pp. 1785–1797, New York, NY, USA, 2021. Association for Computing Machinery. ISBN 9781450383127. doi: 10.1145/3442381.3450078. URL https://doi.org/10.1145/3442381.3450078.

Wang, Z., Zhao, L., Jiang, B., Zhou, G., Zhu, X., and Gai, K. Cold: Towards the next generation of pre-ranking system, 2020.

Xia, X., Eksombatchai, P., Pancha, N., Badani, D. D., Wang, P.-W., Gu, N., Joshi, S. V., Farahpour, N., Zhang, Z., and Zhai, A. Transact: Transformer-based realtime user action model for recommendation at pinterest. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, pp. 5249–5259, New York, NY, USA, 2023. Association for Computing Machinery. ISBN 9798400701030. doi: 10.1145/3580305.3599918. URL https://doi.org/10.1145/3580305.3599918.

Xiao, J., Ye, H., He, X., Zhang, H., Wu, F., and Chua, T.-S. Attentional factorization machines: Learning the weight of feature interactions via attention networks. In *Proceedings of the 26th International Joint Conference on Artificial Intelligence*, IJCAI'17, pp. 3119–3125. AAAI Press, 2017. ISBN 9780999241103.

Xiong, R., Yang, Y., He, D., Zheng, K., Zheng, S., Xing, C., Zhang, H., Lan, Y., Wang, L., and Liu, T.-Y. On layer normalization in the transformer architecture. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

Yang, J., Yi, X., Zhiyuan Cheng, D., Hong, L., Li, Y., Xiaoming Wang, S., Xu, T., and Chi, E. H. Mixed negative sampling for learning two-tower neural networks in recommendations. In *Companion Proceedings of the Web Conference 2020*, WWW '20, pp. 441–447, 2020. ISBN 9781450370240.

Zhai, J., Lou, Y., and Gehrke, J. Atlas: A probabilistic algorithm for high dimensional similarity search. In *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*, SIGMOD '11, pp. 997–1008, 2011. ISBN 9781450306614.

Zhai, J., Gong, Z., Wang, Y., Sun, X., Yan, Z., Li, F., and Liu, X. Revisiting neural retrieval on accelerators. In *Proceedings of the 29th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, KDD '23, pp. 5520–5531, New York, NY, USA, 2023a. Association for Computing Machinery. ISBN 9798400701030. doi: 10.1145/3580305.3599897. URL https://doi.org/10.1145/3580305.3599897.

Zhai, Y., Jiang, C., Wang, L., Jia, X., Zhang, S., Chen, Z., Liu, X., and Zhu, Y. Bytetransformer: A high-performance transformer boosted for variable-length inputs. In *2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, pp. 344–355, Los Alamitos, CA, USA, may 2023b. IEEE Computer Society. doi: 10.1109/IPDPS54959.2023.00042. URL https://doi.ieeecomputersociety.org/10.1109/IPDPS54959.2023.00042.

Zhang, B., Luo, L., Liu, X., Li, J., Chen, Z., Zhang, W., Wei, X., Hao, Y., Tsang, M., Wang, W., Liu, Y., Li, H., Badr, Y., Park, J., Yang, J., Mudigere, D., and Wen, E. Dhen: A deep and hierarchical ensemble network for large-scale click-through rate prediction, 2022.

Zhao, Z., Yang, Y., Wang, W., Liu, C., Shi, Y., Hu, W., Zhang, H., and Yang, S. Breaking the curse of quality saturation with user-centric ranking, 2023.

Zhou, G., Zhu, X., Song, C., Fan, Y., Zhu, H., Ma, X., Yan, Y., Jin, J., Li, H., and Gai, K. Deep interest network for click-through rate prediction. KDD '18, 2018.

Zhou, K., Wang, H., Zhao, W. X., Zhu, Y., Wang, S., Zhang, F., Wang, Z., and Wen, J.-R. S3-rec: Self-supervised learning for sequential recommendation with mutual information maximization. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management*, CIKM '20, pp. 1893–1902, New York, NY, USA, 2020. Association for Computing Machinery. ISBN 9781450368599. doi: 10.1145/3340531.3411954. URL https://doi.org/10.1145/3340531.3411954.

Zhuo, J., Xu, Z., Dai, W., Zhu, H., Li, H., Xu, J., and Gai, K. Learning optimal tree models under beam search. In *Proceedings of the 37th International Conference on Machine Learning*, ICML'20. JMLR.org, 2020.

A. Notations

We summarize key notations used in this paper in Table 7.

Symbol	Description
$\Phi_i(t_j)$	The i-th training example emitted at time j.
n (n_i)	Number of tokens in user history (of user/sample i).
m	Number of candidates considered in a recommendation system's ranking stage.
$s_u(n_i)$, $\hat{s}_u(n_i)$	Sampling rate for user i.
N	$\max_i n_i$
$x_0, \ldots, x_{n-1}$	List of input tokens.
$y_0, \ldots, y_{n-1}$	List of output tokens.
$t_0, \ldots, t_{n-1}$	List of timestamps corresponding to when $x_0, \ldots, x_{n-1}$ were observed.
$a_0, \ldots, a_{n-1}$	List of user actions (e.g., like, image view).
$m_0, \ldots, m_{n-1}$	List of binary masks that we apply to $y_0, \ldots, y_{n-1}$.
$\mathbb{X}$, $\mathbb{X}_c$	Vocabulary of all input/output tokens and its content subset.
u_t	User representation at time t.
d	Model dimension (embedding dimension).
d_{qk}	Attention dimension size in HSTU and Transformers.
d_{ff}	Hidden dimension size in feedforward layers of Transformers.
h	Number of attention heads.
l	Number of layers in HSTU. For Transformers, attention and pointwise feedforward layers together constitute a layer.
α	Hyperparameter controlling sparsity in the *Stochastic Length* algorithm used in HSTU.
$\Gamma(\cdot, L)$	A function that selects a subsequence of length L in *Stochastic Length* (SL).
b_m	Microbatch size, in the M-FALCON algorithm.
R	Register size.
E, F, G, H	Categorical features in Figure 2 (see (Mudigere et al., 2022)).

Table 7. Table of Notations

B. Synthetic Data

As previously discussed in Section 3.1, standard softmax attention, due to its normalization factor, makes it challenging to capture intensity of user preferences which is important for user representation learning. This aspect is important in recommendation scenarios as the system may need to predict the intensity of engagements (e.g., number of future positive actions on a particular topic) in addition to the relative ordering of items.

To understand this behavior, we construct synthetic data following a Dirichlet Process that generates streaming data over a dynamic set of vocabulary. Dirichlet Process captures the behavior that 'rich gets richer' in user engagement histories. We setup the synthetic experiment as follows:

- We randomly assign each one of 20,000 item ids to exactly one of 100 categories.
- We generate 1,000,000 records of length 128 each, with the first 90% being used for training and the final 10% used for testing. To simulate the streaming training setting, we make the initial 40% of item ids available initially and the rest available progressively at equal intervals; i.e., at record 500,000, the maximum id that can be sampled is $(40\% + 60\% * 0.5) * 20,000 = 14,000$.
- We randomly select up to 5 categories out of 100 for each record and randomly sample a prior H_c over these 5 categories. We sequentially sample category for each position following a Dirichlet process over possible categories as follows:
  - for $n > 1$:
    - with probability $\alpha/(\alpha + n - 1)$, draw category c from H_c.
    - with probability $n_c/(\alpha + n - 1)$, draw category c, where n_c is the number of previous items with category c.
    - randomly sample an assigned item matching category c subject to streaming constraints.

  where α is uniformly sampled at random from $(1.0, 500.0)$.

The results can be found in Table 1. We always ablate $\text{rab}^{p,t}$ for HSTU as this dataset does not have timestamps. We observe HSTU increasing Hit Rate@10 by more than 100% relative to standard Transformers. Importantly, replacing HSTU's

Metric Name	Sampler Type		
	Greedy	Weighted	Random
Main Engagement Metric	0.495	0.494	0.495
Main Consumption Metric	0.792	0.789	0.791

Table 8. Impact of Sampler $\Gamma(S, L)$ on model quality, measured by Normalized Entropy (NE).

pointwise attention mechanism with softmax ("HSTU w/ Softmax") also leads to significant reduction in hit rate, verifying the importance of pointwise attention-like aggregation mechanisms.

C. DLRM Baselines

We give a high level description of the DLRM baselines used in Section 4.

Ranking Setting. The baseline ranking model, as described in (Mudigere et al., 2022), employs approximately one thousand dense features and fifty sparse features. We incorporated various modeling techniques such as Mixture of Experts (Ma et al., 2018), Deep & Cross Network (Wang et al., 2021), target-aware pairwise attention (Zhou et al., 2018), and residual connection over special interaction layers (Zhang et al., 2022).

Retrieval Setting. The baseline retrieval model employs a standard two-tower neutral retrieval setting (Covington et al., 2016) with mixed in-batch and out-of-batch sampling. The input feature set consists of both high cardinality sparse features (e.g., item ids, user ids) and low cardinality sparse features (e.g. languages, topics, interest entities). A stack of feed forward layers with residual connections (He et al., 2015) is used to compress the input features into user and item embeddings.

D. Stochastic Length

D.1. Selection of Sampling Technique Γ

In Equation (4), we introduced a function $\Gamma(S, L)$ which samples a sequence S to length L in order to increase sparsity. Our empirical results indicate that properly designing Γ can improve model quality. During the selection of Γ, we compute a metric $f_i = t_n - t_i$ which corresponds to the amount of time elapsed since the user interacted with item x_i. We conduct offline experiments with the following samplers:

- **Greedy Sampler** – Samples L items with smallest values of f_i from S
- **Random Sampler** – Samples L items from S randomly
- **Feature-Weighted Sampler** – Samples L items from S according to a weighted distribution $1 - f_{n,i}/(\sum_{j=1}^{L} f_{j,i})$

During our offline experiments, the feature-weighted sampler resulted in the best model quality, as shown in Table 8.

D.2. Impact of Stochastic Length on Sequence Sparsity

In Table 2, we show the impact of Stochastic Length on sequence sparsity for a representative industry-scale configuration with 30-day user engagement history. The sequence sparsity is defined as one minus the ratio of the average sequence length of all samples divided by the maximum sequence length. To better characterize the computational cost of sparse attentions, we also define *s2*, which is defined as one minus the sparsity of the attention matrix. For reference, we present the results for 60-day and 90-day user engagement history in Table 9 and Table 10, respectively.

D.3. Comparisons Against Sequence Length Extrapolation Techniques

We conduct additional studies to verify that *Stochastic Length* is competitive against existing techniques for sequence length extrapolation used in language modeling. Many existing methods perform sequence length extrapolation through modifications of RoPE (Su et al., 2023). To compare against existing methods, we train a HSTU variant (HSTU-RoPE) with no relative attention bias and rotary embeddings.

We evaluate the following sequence length extrapolation methods on HSTU-RoPE:

- **Zero-Shot** - Apply NTK-Aware RoPE (Peng et al., 2024) before directly evaluating the model with no finetuning;

Table 9. Impact of Stochastic Length (SL) on sequence sparsity, over a 60d user engagement history.

Alpha	Max Sequence Length							
	1,024		2,048		4,096		8,192	
	sparsity	s2	sparsity	s2	sparsity	s2	sparsity	s2
1.6	71.5%	89.4%	75.8%	92.3%	79.4%	94.7%	83.8%	97.3%
1.7	57.3%	77.6%	60.6%	79.8%	67.3%	86.6%	74.5%	93.3%
1.8	37.5%	56.2%	42.6%	62.1%	51.9%	74.2%	62.6%	85.5%
1.9	15.0%	25.2%	17.7%	29.0%	29.6%	47.5%	57.8%	80.9%
2.0	1.2%	1.7%	2.5%	3.5%	18.9%	30.8%	57.6%	80.6%

Table 10. Impact of Stochastic Length (SL) on sequence sparsity, over a 90d user engagement history.

Alpha	Max Sequence Length							
	1,024		2,048		4,096		8,192	
	sparsity	s2	sparsity	s2	sparsity	s2	sparsity	s2
1.6	68.0%	85.0%	74.6%	90.8%	78.6%	93.5%	83.5%	97.3%
1.7	56.3%	76.1%	61.2%	80.6%	67.5%	87.0%	74.3%	93.3%
1.8	38.9%	58.3%	42.0%	61.3%	50.4%	72.4%	61.0%	84.4%
1.9	16.2%	27.3%	17.3%	28.6%	27.2%	44.4%	54.3%	77.8%
2.0	0.9%	1.2%	1.6%	2.1%	13.5%	22.5%	54.0%	77.4%

- **Fine-tune** - Finetune the model for 1000 steps after applying NTK-by-parts (Peng et al., 2024).

We evaluate the following sequence length extrapolation methods on HSTU (includes relative attention bias, no rotary embeddings):

- **Zero-Shot** - Clamp the relative position bias according to the maximum training sequence length, directly evaluate the model (Raffel et al., 2020; Press et al., 2022);
- **Fine-tune** - Clamp the relative position bias according to the maximum training sequence length, fine-tune the model for 1000 steps before evaluating the model.

In Table 11, we report the NE difference between models with induced data sparsity during training (Stochastic Length, zero-shot, fine-tuning) and models trained on the full data. We define the sparsity for zero-shot and fine-tuning techniques to be the average sequence length during training divided by the max sequence length during evaluation. All zero-shot and fine-tuned models are trained on 1024 sequence length data and are evaluated against 2048 and 4096 sequence length data. In order to find an appropriate *Stochastic Length* baseline for these techniques, we select *Stochastic Length* settings which result in the same data sparsity metrics.

We believe that zero-shot and fine-tuning approaches to sequence length extrapolation are not well-suited for recommendation scenarios that deal with high cardinality ids. Empirically, we observe that *Stochastic Length* significantly outperforms fine-tuning and zero-shot approaches. We believe that this could be due to our large vocabulary size. Zero-shot and

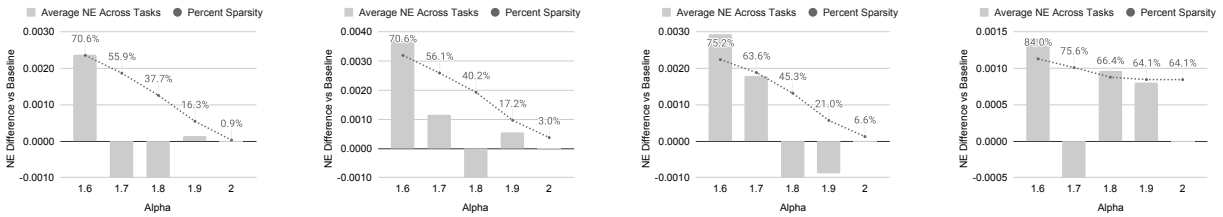


Figure 8. Impact of *Stochastic Length* (SL) on ranking model metrics. Left to right: $n = [1024, 2048, 4096, 8192]$ (n is after interleaving algorithm as discussed in Section 2.2 to enable target-aware cross attention in causal-masked settings).

Evaluation Strategy	Average NE Difference vs Full Sequence Baseline		
	Model Type	2048 / 52% Sparsity	4096 / 75% Sparsity
Zero-shot	HSTU (Raffel et al., 2020)	6.46%	10.35%
	HSTU-RoPE (Peng et al., 2024)	7.51%	11.27%
Fine-tune	HSTU (Raffel et al., 2020)	1.92%	2.21%
	HSTU-RoPE (Peng et al., 2024)	1.61%	2.19%
Stochastic Length (SL)	HSTU	**0.098%**	**0.64%**

Table 11. Comparisons of *Stochastic Length* (SL) vs existing Length Extrapolation methods.

fine-tuning approaches fail to learn good representations for older ids, which could hurt their ability to fully leverage the information contained in longer sequences.

E. Evaluation of Inference Throughput: GRs w/ M-FALCON vs DLRMs

As discussed in Section 3.4, M-FALCON handles b_m candidates in parallel to amortize computation costs across all candidates at inference time. To understand our design, we compare the throughput (i.e., number of candidates scored per second) of GRs and DLRMs based on the same hardware setups. As shown in Figure 9 and Figure 10, GRs' throughput scales in a sublinear way based on the number of ranking-stage candidates (m), up to a certain region – $m = 2048$ in our case study – due to cost amortization. Due to attention complexity scaling as $O((n + b_m)^2)$, leveraging multiple microbatches by itself improves throughput. Caching further eliminates redundant linear and attention computations on top of microbatching. The two combined resulted in up to 1.65x speedups relative to the $m = 1024$ baseline using a single microbatch, as shown in Figure 10. Overall, utilizing M-FALCON, HSTU-based GRs outperform DLRMs in terms of throughput on a large-scale production setup by up to 2.48x, despite GRs being 285x more complex in terms of FLOPs.

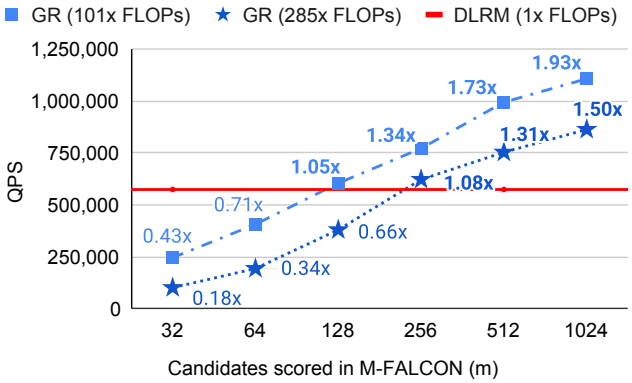


Figure 9. End-to-end inference throughtput: DLRMs vs GRs (w/ M-FALCON) in large-scale industrial settings.

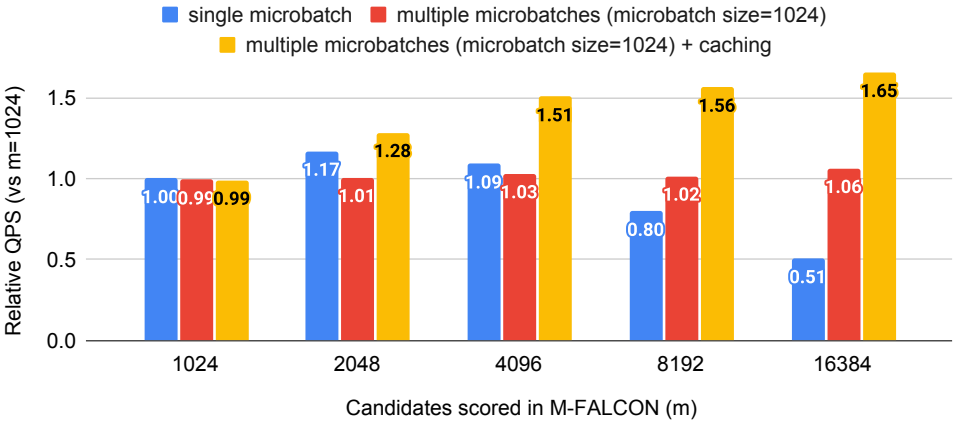


Figure 10. End-to-end inference throughtput: M-FALCON throughput scaling, on top of the 285x FLOPs GR model, in large batch settings where m (total number of candidates) ranges from 1024 to 16384 and $b_m = 1024$.